

Veeva Previous-Year Java MCQ Practice

This sheet combines the previous-year questions from yesterday and today's OOP set. Use it for revision before Round 1 and Round 2.

Set 1: Previous-Year Java MCQs

1. How is encapsulation primarily achieved in Java?

Answer: B - By using private variables and public getter/setter methods

2. Which statement about interfaces in Java is correct?

Answer: B - Interfaces support multiple inheritance in Java

3. When does overloading not occur?

Answer: B - Same method name and same signature

4. Which statement about inheritance is true?

Answer: B - A subclass inherits methods from superclass

5. What does equals() usually compare?

Answer: B - Object content

6. Why override hashCode() when overriding equals()?

Answer: B - To maintain hash table contract

7. Which is true about Object class?

Answer: A - All classes inherit from Object

8. What is visibility of protected?

Answer: C - Same package and subclasses

9. What is visibility of private?

Answer: B - Same class only

10. Which is true about multiple inheritance in Java?

Answer: A - A class can implement multiple interfaces

11. What is the output?

```
Integer num1 = 100;
Integer num2 = 100;
Integer num3 = 500;
Integer num4 = 500;

System.out.println(num1 == num2);
System.out.println(num3 == num4);
```

Answer: B - true false

Reason: Integer values from -128 to 127 are cached. 100 uses cache, 500 does not.

12. What is the output?

```
byte[] arr = {97, 98, 99, 100, 101};
String str = new String(arr);
System.out.println(str);
```

Answer: abcde

13. What is the output?

```
String str = "Java Programming";  
char ch = str.charAt(2);  
System.out.println(ch);
```

Answer: v

Set 1 Coding Questions With Java Code

Coding 1: Closest Number To Target

Problem: Given an array and target K, return the number with the smallest absolute difference from K. If there is a tie, choose the greater number.

```
class Main {  
    public static void main(String[] args) {  
        int[] arr = {9, 11, 5, 3, 25, 18};  
        int k = 6;  
  
        int closest = arr[0];  
        int minDiff = Math.abs(arr[0] - k);  
  
        for (int i = 1; i < arr.length; i++) {  
            int diff = Math.abs(arr[i] - k);  
  
            if (diff < minDiff || (diff == minDiff && arr[i] > closest)) {  
                minDiff = diff;  
                closest = arr[i];  
            }  
        }  
  
        System.out.println(closest);  
    }  
}
```

Time Complexity: O(n)

Space Complexity: O(1)

Coding 2: Validate Subsequence

Problem: Check whether sequence is a subsequence of array. Elements must appear in the same order, but not necessarily continuously.

```
class Main {  
    public static void main(String[] args) {  
        int[] array = {5, 1, 22, 25, 6, -1, 8, 10};  
        int[] sequence = {1, 6, -1, 10};  
  
        int i = 0;  
  
        for (int x : array) {  
            if (i < sequence.length && x == sequence[i]) {  
                i++;  
            }  
        }  
  
        System.out.println(i == sequence.length);  
    }  
}
```

Time Complexity: O(n)

Space Complexity: O(1)

Coding 3: First Non-Repeating Character

Problem: Find the first character that appears only once. If none exists, print -1.

Frequency array version for lowercase English letters:

```
class Main {
    public static void main(String[] args) {
        String s = "loveleetcode";

        int[] freq = new int[26];

        for (int i = 0; i < s.length(); i++) {
            freq[s.charAt(i) - 'a']++;
        }

        for (int i = 0; i < s.length(); i++) {
            if (freq[s.charAt(i) - 'a'] == 1) {
                System.out.println(s.charAt(i));
                System.out.println(i);
                return;
            }
        }

        System.out.println(-1);
    }
}
```

HashMap version for general characters:

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        String s = "loveleetcode";

        HashMap<Character, Integer> map = new HashMap<>();

        for (char ch : s.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);

            if (map.get(ch) == 1) {
                System.out.println(ch);
                System.out.println(i);
                return;
            }
        }

        System.out.println(-1);
    }
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$ for frequency array, $O(k)$ for HashMap

Set 2: OOP Java MCQs

1. When a class implements an interface, which of the following must be implemented?

Answer: A - Only abstract methods

2. What happens if a method in a subclass has the same name and parameters as a method in its superclass?

Answer: B - The subclass method is executed

3. Which of the following correctly describes a Java interface?

Answer: D - It can only contain abstract methods

Note: This is the old/basic MCQ answer. Modern Java interfaces can also have default and static methods.

4. What is the purpose of the Serializable interface in Java?

Answer: B - To allow objects to be serialized

5. How do you prevent a method from being overridden in a subclass?

Answer: A - Declare it as final

6. Which statement about multiple inheritance of interfaces is true?

Answer: A - It is allowed in Java

7. Which keyword is used to declare an inner class?

Answer: D - class

8. What is the result of attempting to cast an object to a class that it does not inherit from?

Answer: B - Run-time exception, usually ClassCastException

9. In object serialization, what is serialVersionUID?

Answer: A - A unique identifier for each class version

10. What is the purpose of a marker interface in Java?

Answer: B - To mark a class for specific behavior

11. Which statement about static members is true?

Answer: B - Static members can be accessed without creating an instance of a class

12. What is the output?

```
class Test {
    public final int a;
}

class Example {
    public static void main(String args[]) {
        Test obj = new Test();
        System.out.println(obj.a);
    }
}
```

Answer: C - Compile time error: variable is not initialized

Reason: a is a blank final variable and must be initialized.

13. What is the output?

```
class Example {
    private int x;

    public static void main(String args[]) {
        Example obj = new Example();
    }

    public void Example(int x) {
        System.out.println(x);
    }
}
```

Answer: D - No output / blank screen

Reason: public void Example(int x) is a normal method, not a constructor.

14. What is the output?

```
class Example {
    private int x;

    public static void main(String args[]) {
        Example obj = new Example(5);
    }
}
```

```

    }

    public Example(int x) {
        System.out.println("x = " + x);
    }

    public void Example(int x) {
        System.out.println(x);
    }
}

```

Answer: A - x = 5

Reason: public Example(int x) is the constructor. The void version is a normal method.

15. What is the output?

```

class Test {
    public static void main(String args[]) {
        String str1 = new String("Hello World");
        String str2 = new String("Hello World");

        String str3 = "Hello World";
        String str4 = "Hello World";

        int a = 0, b = 0, c = 0;

        if (str3 == str4)
            a = 1;
        else
            a = 2;

        if (str1.equals(str3))
            b = 1;
        else
            b = 2;

        if (str1 == str4)
            c = 1;
        else
            c = 2;

        System.out.println("a= " + a + " b= " + b + " c= " + c);
    }
}

```

Answer: D - a= 1 b= 1 c= 2

Reason: str3 and str4 point to the same string pool object. equals compares content. str1 and str4 are different references.

16. How many String objects are created in the above example?

Answer: C - 3

Reason: one string pool literal plus two new String objects.

17A. What is the output?

```

import java.util.HashMap;
import java.util.Map;

public class MyClass {
    public static void main(String args[]) {
        Map<String, String> hashMap = new HashMap<String, String>();
        hashMap.put(new String("a"), "audi");
        hashMap.put(new String("a"), "ferrari");
        System.out.println(hashMap);
    }
}

```

Answer: {a=ferrari}

Reason: HashMap compares keys using equals() and hashCode(), so the second value replaces the first.

17B. What is the output?

```

import java.util.IdentityHashMap;

```

```
import java.util.Map;

public class MyClass {
    public static void main(String args[]) {
        Map<String, String> identityHashMap = new IdentityHashMap<String, String>();
        identityHashMap.put(new String("a"), "audi");
        identityHashMap.put(new String("a"), "ferrari");
        System.out.println(identityHashMap);
    }
}
```

Answer: Both entries remain, for example {a=audi, a=ferrari}. Order may vary.

Reason: IdentityHashMap compares keys using ==, not equals().